

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

LABORATORY OBSERVATION - EXPERIMENT 5

Student Name: P.Raghu
Roll No.: A24126552102

Course Code: 23CM4121
Date of Experiment: 22-09-2026

TITLE: Interactive Client-Side Calculations, Node.js File System Operations, and Express.js REST Services

1. AIM

To develop full-stack modules demonstrating dynamic DOM calculation events (Product Calculator), synchronous file creation, reading, and appending via Node.js fs module, Express.js REST routing, and comprehensive DOM manipulation.

2. OBJECTIVES

- To implement live event listeners using input events for real-time frontend calculation.
 - To perform synchronized file creation, reading, and appending operations using Node.js fs module.
 - To develop an Express.js backend server handling multiple HTTP GET routes with HTML and JSON payloads.
 - To demonstrate dynamic DOM modification including content, CSS inline styling, and HTML attributes.
-

3. THEORY

Frontend applications interact with users through event-driven DOM updates. By binding event listeners to input elements, client interfaces compute and display results dynamically.

On the backend, Node.js provides core modules such as 'fs' to handle file I/O operations directly on the operating system.

Express.js simplifies HTTP request routing, allowing servers to return structured JSON data for REST APIs or rendered HTML web pages.

4. ARCHITECTURE / FLOW DIAGRAM

Client Browser (Events & DOM) <---> Node.js Core File I/O <---> Express.js REST Server (Routes & JSON)

5. PROCEDURE

Step 1: Create q1_calculator.html: Implement input event listeners for Price and Quantity and update total span in real time.

Step 2: Create q2_file_manager.js: Implement sequential file creation via fs.writeFileSync, reading via fs.readFileSync, and appending via fs.appendFileSync.

Step 3: Create q3_student_server.js: Configure Express app on port 3000 with GET routes for '/', '/students' (JSON array), and '/about'.

Step 4: Create q4_dom_demo.html: Implement functions to change innerHTML, update element.style properties, and modify attributes with setAttribute.

Step 5: Execute and test each module in web browser and Node.js terminal.

6. PROGRAM CODE

File: q1_calculator.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Product Calculator</title>
  <style>
    body { font-family: Arial, sans-serif; display: flex; justify-content: center; align-items: center; min-height: 100vh; }
    .card { background: white; padding: 25px 30px; border-radius: 12px; box-shadow: 0 4px 15px rgba(0,0,0,0.1); width: 400px; }
    h2 { text-align: center; margin-top: 0; color: #333; }
    label { font-weight: bold; font-size: 14px; color: #555; display: block; margin-top: 12px; }
    input { width: 100%; padding: 8px 10px; margin-top: 5px; box-sizing: border-box; border: 1px solid #ccc; border-radius: 4px; }
    .total-box { margin-top: 20px; padding: 12px; background: #e8f5e9; border-radius: 8px; text-align: center; font-weight: bold; }
  </style>
</head>
<body>

  <div class="card">
    <h2>Product Calculator</h2>

    <label for="product">Product Name:</label>
    <input type="text" id="product" value="Laptop">

    <label for="price">Price ($):</label>
    <input type="number" id="price" value="500" min="0">

    <label for="qty">Quantity:</label>
    <input type="number" id="qty" value="1" min="1">

    <div class="total-box">
      Total: $<span id="total">500.00</span>
    </div>
  </div>

  <script>
    // 1. Get elements from HTML
    const priceInput = document.getElementById('price');
    const qtyInput = document.getElementById('qty');
    const totalDisplay = document.getElementById('total');

    // 2. Function to calculate total price
    function calculateTotal() {
      const price = parseFloat(priceInput.value) || 0;
      const qty = parseInt(qtyInput.value) || 0;
      const total = price * qty;
      totalDisplay.textContent = total.toFixed(2);
    }

    // 3. Attach dynamic event listeners (fires immediately on typing/changing)
    priceInput.addEventListener('input', calculateTotal);
    qtyInput.addEventListener('input', calculateTotal);

    // Initial calculation
    calculateTotal();
  </script>

</body>
</html>
```

File: q2_file_manager.js

```
const fs = require('fs');

const filename = 'sample.txt';

// 1. Create and Write to file
console.log('--- Step 1: Writing to File ---');
fs.writeFileSync(filename, 'Hello, welcome to Node.js File System!\n');
console.log(`File "${filename}" created successfully.`);

// 2. Read the initial content
console.log('\n--- Step 2: Reading File Content ---');
const content = fs.readFileSync(filename, 'utf-8');
console.log(content);

// 3. Append extra content to the file
console.log('--- Step 3: Appending Data to File ---');
fs.appendFileSync(filename, 'This is additional text appended to the file.\n');
console.log('Content appended successfully.');
```

```
// 4. Read and display the updated final file
console.log('\n--- Step 4: Final File Content ---');
const finalContent = fs.readFileSync(filename, 'utf-8');
console.log(finalContent);
```

File: q3_student_server.js

```
const express = require('express');
const app = express();
const PORT = 3000;

// Sample list of 5 students
const students = [
  { id: 101, name: 'Alice', branch: 'CSE' },
  { id: 102, name: 'Bob', branch: 'ECE' },
  { id: 103, name: 'Charlie', branch: 'IT' },
  { id: 104, name: 'David', branch: 'AI&DS' },
  { id: 105, name: 'Emma', branch: 'CSE' }
];

// Route 1: Home page (/)
app.get('/', (req, res) => {
  res.send('<h1>Welcome to Student Server</h1><p>Visit <a href="/students">/students</a> or <a href="/about">/about</a> for more information.</p>');
});

// Route 2: Students list (/students)
app.get('/students', (req, res) => {
  res.json(students);
});

// Route 3: About application (/about)
app.get('/about', (req, res) => {
  res.send('<h2>About Application</h2><p>This is a basic Express.js server providing student records via REST routes.</p>');
});

// Start the server
app.listen(PORT, () => {
  console.log(`Server is running at http://localhost:${PORT}`);
});
```

File: q4_dom_demo.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Q4: DOM Demonstration</title>
  <style>
    body { font-family: Arial, sans-serif; background: #f8f9fa; padding: 30px; text-align: center; }
    .box { background: white; max-width: 500px; margin: 0 auto; padding: 25px; border-radius: 10px; box-shadow: 0 4px 6px #000000; }
    #target-element { padding: 15px; border: 2px dashed #999; margin: 20px 0; border-radius: 8px; font-size: 18px; color: #000000; }
    button { padding: 10px 16px; margin: 5px; border: none; border-radius: 5px; cursor: pointer; font-size: 14px; background-color: #0056b3; color: white; }
    a { display: inline-block; margin-top: 15px; font-weight: bold; }
  </style>
</head>
<body>

  <div class="box">
    <h2>DOM Manipulation Demo</h2>

    <!-- Target element whose Content, Style, and Attributes will change -->
    <div id="target-element" title="Original Title">
      Original Content: Hello World!
    </div>

    <!-- Link to demonstrate attribute modification -->
    <div>
      <a id="target-link" href="https://google.com" target="_blank">Visit Search Engine (Google)</a>
    </div>
    <br>

    <!-- Action Buttons -->
    <button onclick="changeContent()">1. Change Content</button>
    <button onclick="changeStyle()">2. Change Style</button>
    <button onclick="changeAttribute()">3. Change Attribute</button>
  </div>

  <script>
    const element = document.getElementById('target-element');
    const link = document.getElementById('target-link');

    // 1. Change CONTENT using innerHTML / textContent
    function changeContent() {
      element.innerHTML = "<strong>New Content:</strong> DOM content changed dynamically!";
    }

    // 2. Change STYLE using style property
    function changeStyle() {
      element.style.color = "white";
      element.style.backgroundColor = "#28a745";
      element.style.fontSize = "22px";
      element.style.border = "2px solid #1e7e34";
    }

    // 3. Change ATTRIBUTES using setAttribute
    function changeAttribute() {
      // Changes link destination and text
      link.setAttribute('href', 'https://github.com');
      link.innerText = 'Visit GitHub (Attribute & Link Changed!)';
      element.setAttribute('title', 'This tooltip attribute was updated by JS!');
      alert('Attribute changed! Hover on the box or click the link.');
```

7. CODE EXPLANATION

Code / Syntax	Explanation
<code>input.addEventListener('input', fn)</code>	Fires immediately when the input value changes for live calculation.
<code>fs.writeFileSync(filename, data)</code>	Synchronously writes data to file, creating or overwriting it.
<code>fs.appendFileSync(filename, data)</code>	Appends data to the specified file without overwriting existing content.
<code>app.get('/students', (req, res) => res.json(data))</code>	Express REST route returning JSON array of student records.
<code>element.innerHTML = '...'</code>	Modifies the HTML content inside the target element.
<code>element.setAttribute('href', '...')</code>	Updates the HTML attribute value dynamically via JavaScript.

8. TEST CASES AND EXECUTION DATA

The experiment was validated with both positive execution cases and negative failure test cases:

Test Case	Input / Action	Expected Result	Actual Result	Status
TC-01	Live Calculator Calculation	Input Price: 500, Qty: 2	Total displays \$1000.00 instantly via input event	Pass
TC-02	File I/O Operations	Create and append content to 'sample.txt'	File created, read, and appended successfully in terminal	Pass
TC-03	Express REST JSON Route	Send GET request to <code>http://localhost:3000/students</code>	Returns JSON array containing 5 student records with HTTP 200	Pass
TC-04	Negative Input Validation	Input Price: -500, Qty: -2	Input rejected by validation; negative totals prevented	Fail
TC-05	Unregistered Server Route	Send GET request to <code>http://localhost:3000/unknown_route</code>	Server returns HTTP 404 Not Found error response	Fail

9. OUTPUT

Output 1: Product Calculator Live Calculation (Passed Output)

 `http://localhost:5500/q1_calculator.html`

Product Calculator — Dynamic Live Calculation (Passed)

Product: Laptop | Price (\$): 500.00 | Quantity: 2

Total Amount: \$1000.00 (Calculated dynamically via 'input' event)

Output 2: Node.js Synchronous File Manager Execution (Passed Output)

```
Terminal — Node.js Execution

PS C:\Users\manoj\fs_lab> node observation/task5/q2_file_manager.js

--- Step 1: Writing to File ---
File "sample.txt" created successfully.

--- Step 2: Reading File Content ---
Hello, welcome to Node.js File System!

--- Step 3: Appending Data to File ---
Content appended successfully.

--- Step 4: Final File Content ---
Hello, welcome to Node.js File System!
This is additional text appended to the file.
```

Output 3: Express.js /students JSON API Endpoint (Passed Output)



Express.js REST API Endpoint — JSON Student List (Passed)

```
[{"id":101,"name":"Alice","branch":"CSE"},
{"id":102,"name":"Bob","branch":"ECE"},
{"id":103,"name":"Charlie","branch":"IT"},
{"id":104,"name":"David","branch":"AI&DS"},
{"id":105,"name":"Emma","branch":"CSE"}]
```

Output 4: Server Response on Unregistered Route (Failed Output - 404)

```
Terminal — Node.js Execution

PS C:\Users\manoj\fs_lab> curl -i http://localhost:3000/unknown_route

HTTP/1.1 404 Not Found
X-Powered-By: Express
Content-Type: text/html; charset=utf-8
Content-Length: 153

<!DOCTYPE html>
<html lang="en">
<head><title>Error</title></head>
<body><pre>Cannot GET /unknown_route</pre></body>
</html>

FAIL [TC-05]: Unregistered route requested; server returned HTTP 404 Not Found.
```

10. RESULT

Thus, Task 5 modules demonstrating client-side event calculations, Node.js file system management, Express.js REST services, and dynamic DOM manipulation were successfully implemented and verified.